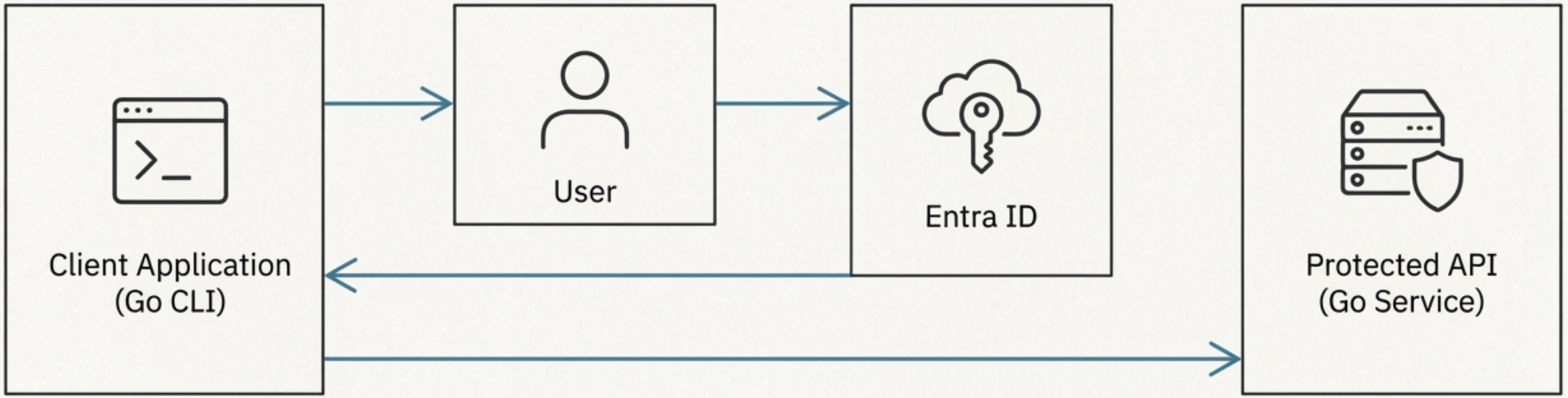
The Full-Stack Security Journey

Securing a Go Application with Entra ID, from Client Interaction to Cryptographic Verification

The Four Actors in Our Security Story



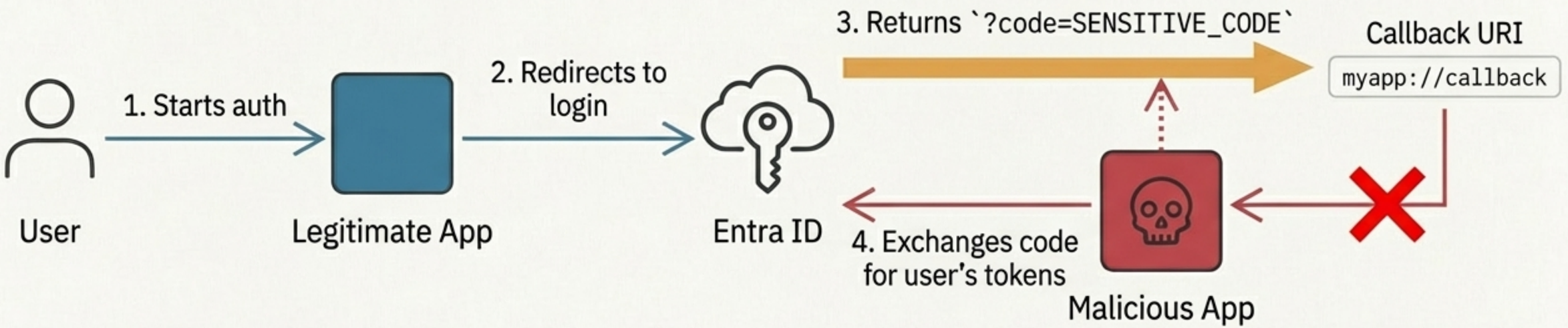
This presentation follows the journey of a request, starting with the Client App and ending with a secure response from the Protected API. We will revisit this map to orient ourselves as we dive into each component.

Part 1: The Client’s Quest for a Token

How does a public client, which cannot store a secret, securely obtain an access token on behalf of a user?

- Public clients include desktop apps, mobile apps, CLIs, and Single-Page Applications (SPAs).
- Their code is distributed and can be decompiled, meaning any embedded ‘client secret’ is easily extracted.
- The traditional Authorization Code flow is vulnerable in this scenario. We need a mechanism that doesn’t rely on a pre-configured secret.

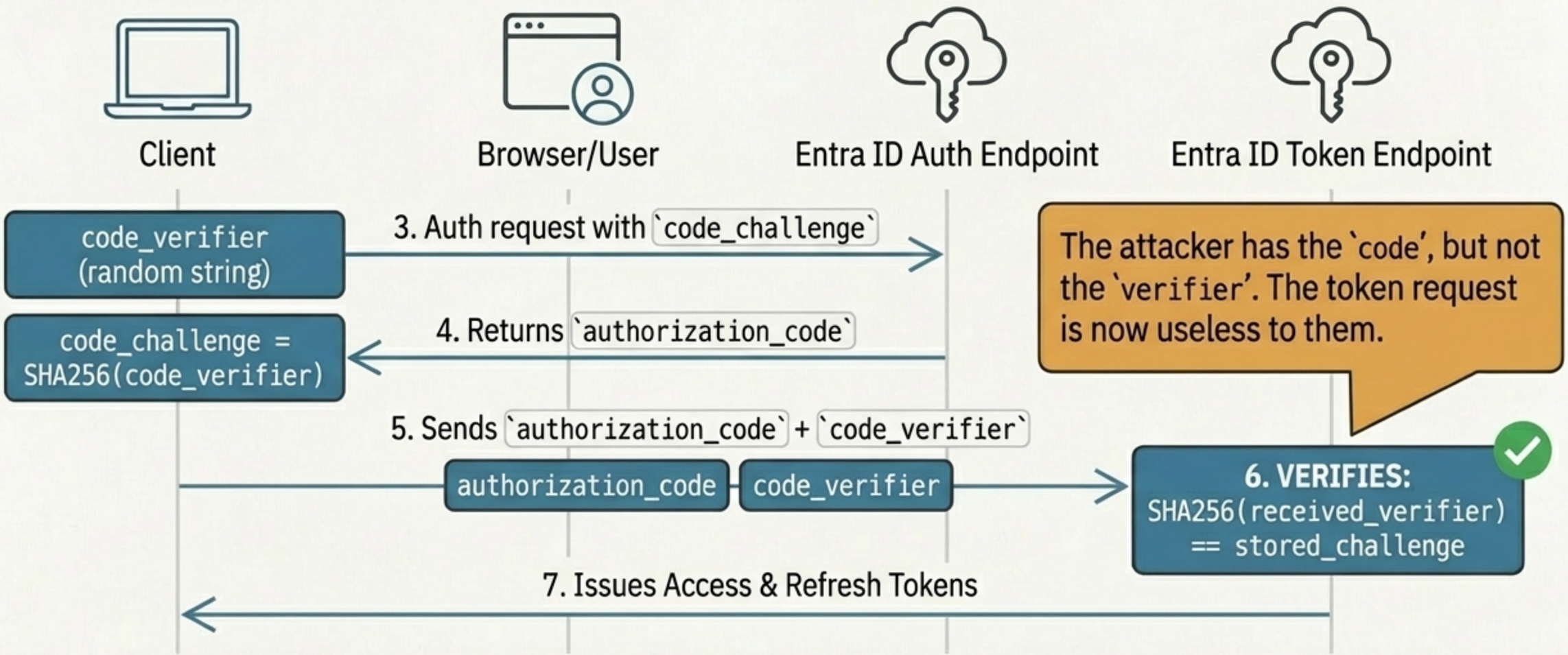
The Vulnerability: Authorization Code Interception



Without PKCE, any application on a user's machine that can intercept the callback URI can steal the authorization code and impersonate the user.

The Solution: Proof Key for Code Exchange (PKCE)

PKCE cryptographically binds the authorization request to the token exchange.



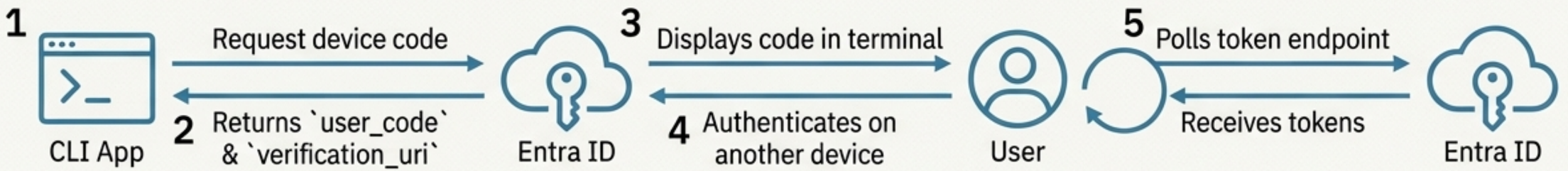
When There’s No Browser: The Device Code Flow

The optimal solution for headless and CLI environments (used by Azure CLI, GitHub CLI, kubectl).

Think of it like activating a new smart TV.

The TV (your CLI) shows a code and a URL.

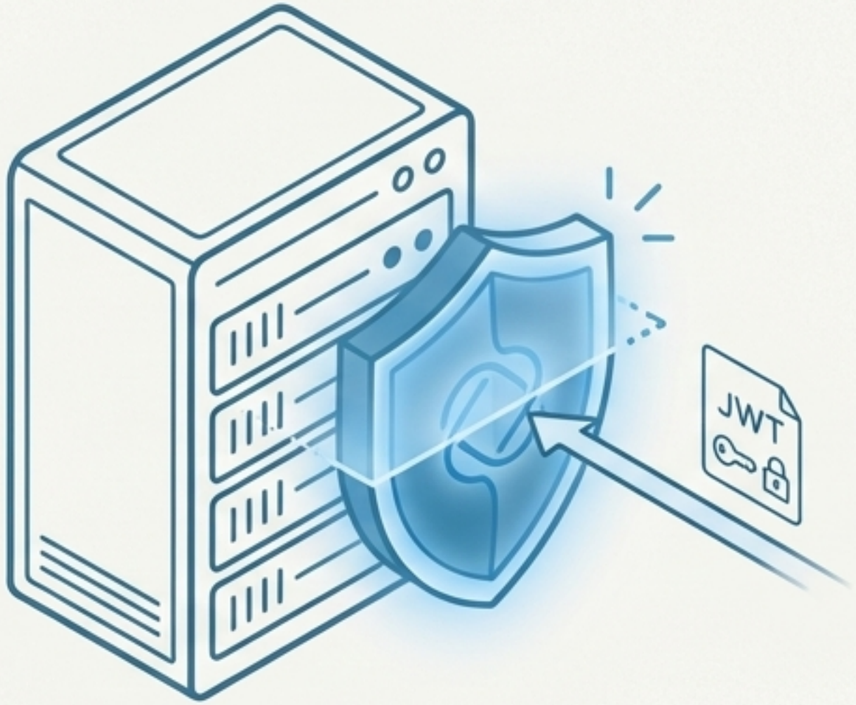
You use your phone (any browser-enabled device) to complete the login, and the TV is then automatically authorized.



This flow fully supports MFA and Conditional Access, unlike the discouraged Username/Password flow.

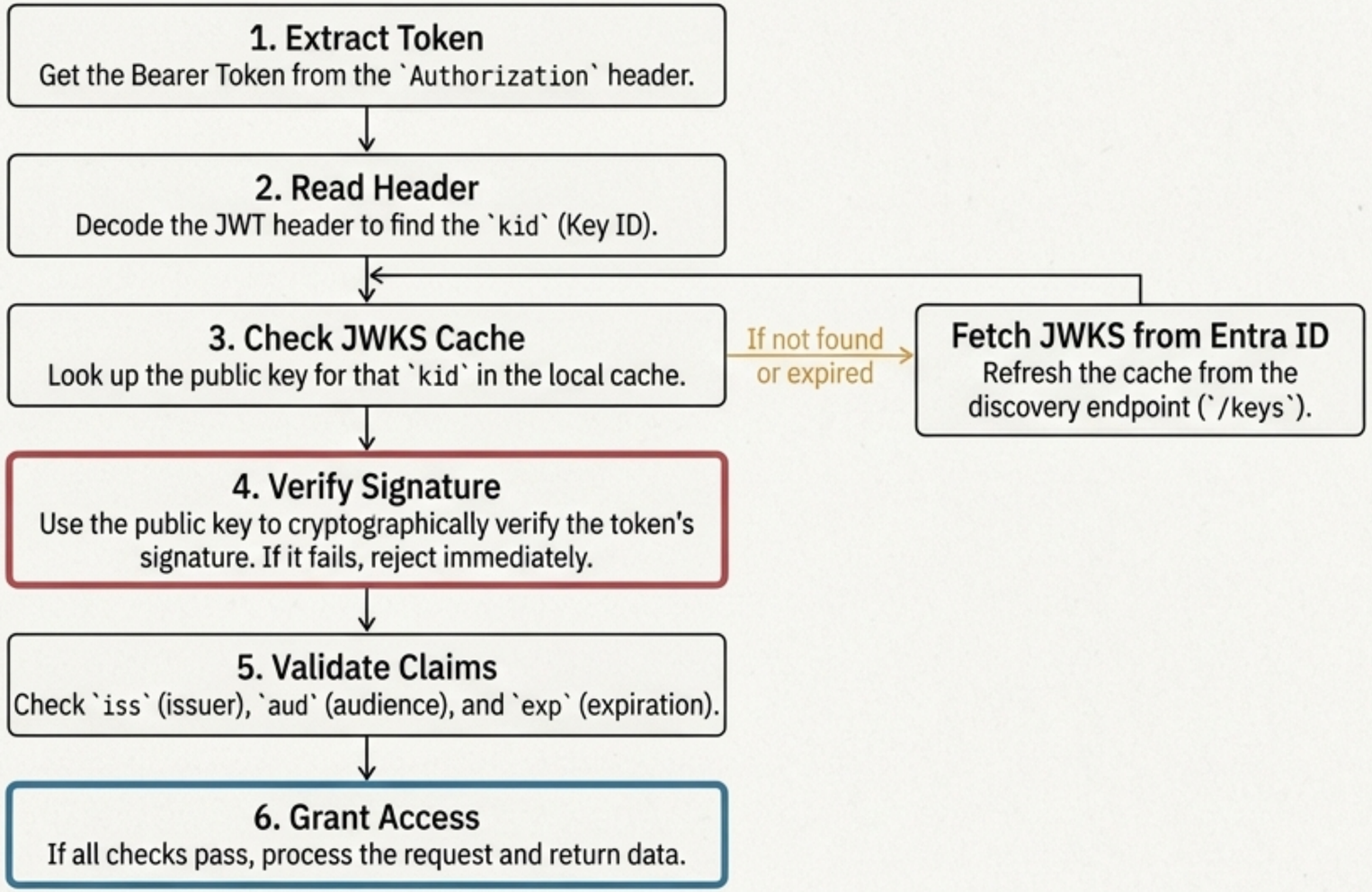
Part 2: The API's Duty to Verify

An application presents a token. How does our API prove it's authentic, untampered, and issued for the correct purpose?



- The client now has an access token. It sends this token in the `Authorization: Bearer <token>` header with every request to our protected API.
- The API must perform a series of checks without constantly asking the Identity Provider. This requires validating the token's cryptographic signature locally.

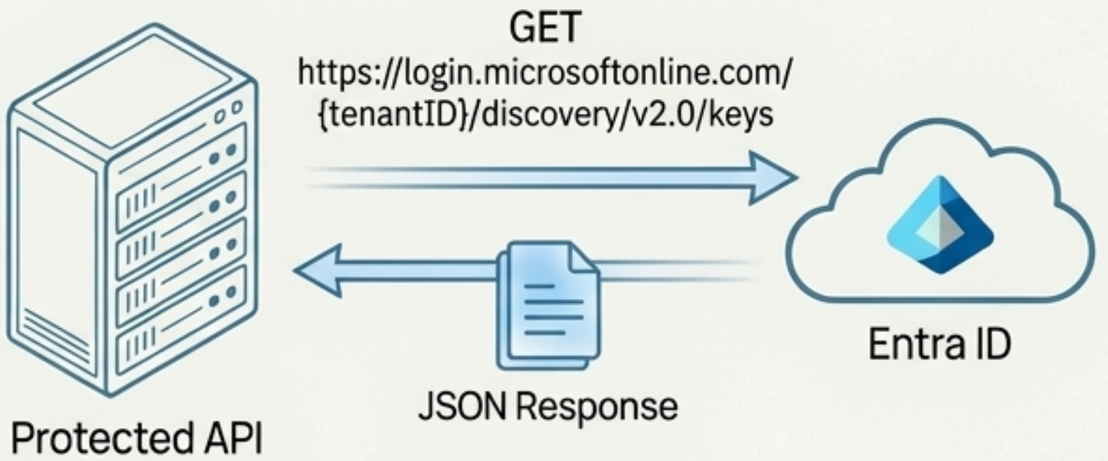
The Anatomy of API Token Validation



Caching the JWKS is critical for performance and allows the API to validate tokens without a network call to Entra ID for every request.

The Source of Truth: JSON Web Key Set (JWKS)

A JWKS is a set of public keys exposed by the Identity Provider at a well-known URL. Each key has a unique Key ID (`kid`). A JWT's header contains the `kid` of the key that was used to sign it, telling the API exactly which public key to use for verification.




```
{
  "keys": [
    {
      "kid": "nOo3ZDrODXEK1jKWhXslHR_KXEg",
      "kty": "RSA",
      "use": "sig",
      "n": "oahUIoWw0K0usKNuOR6...",
      "e": "AQAB"
  ] }
}
```

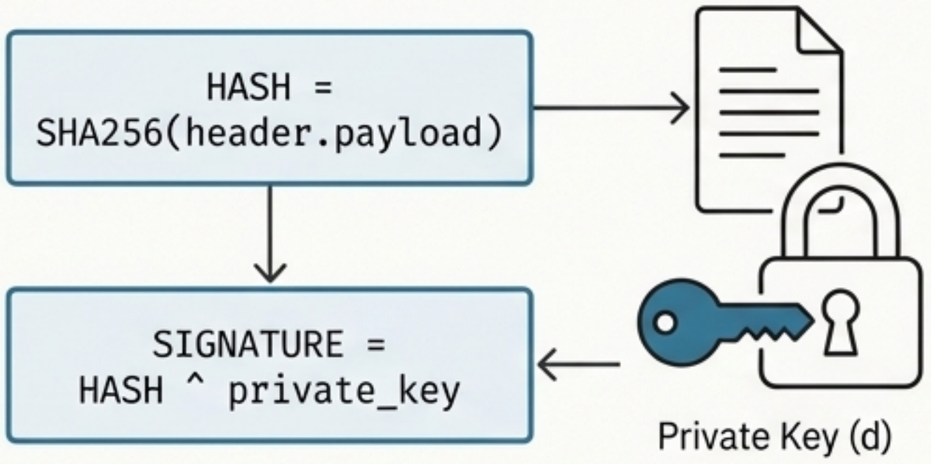
This mechanism enables seamless key rotation. When Entra ID rotates its signing keys, the API automatically fetches the new ones the next time it refreshes its cache.

The Cryptographic Proof: How Signature Verification Works

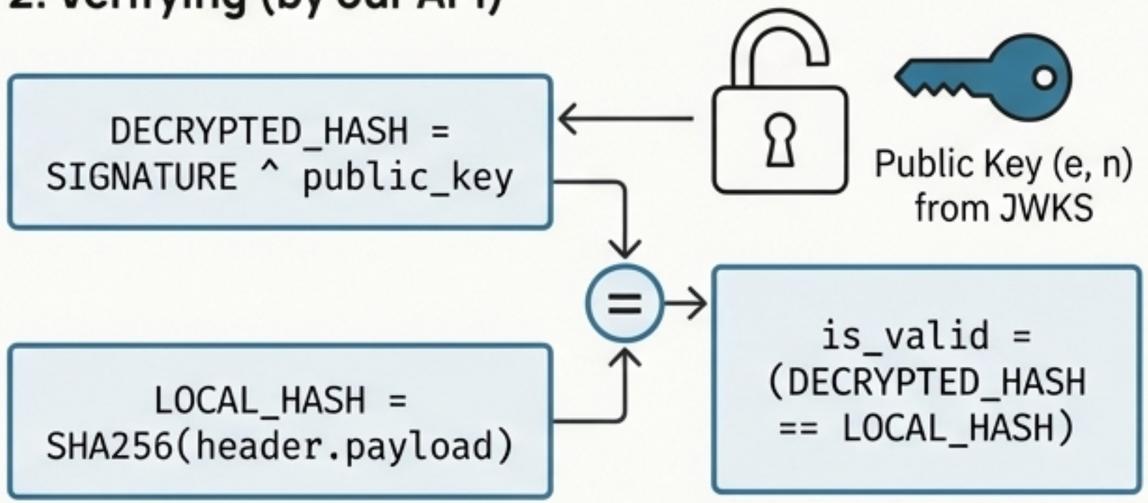
RSA validation relies on the relationship between a private key (`d`) and a public key (`e`).

`(message ^ d) ^ e ≡ message (mod n)`

1. Signing (by Entra ID)



2. Verifying (by our API)



A `crypto/rsa: verification error` means the locally computed hash does not match the hash recovered from the signature. The token has been tampered with, or the wrong public key was used.

Troubleshooting the Dreaded `crypto/rsa: verification error`

When the math is correct, the error is almost always in the data. Here are the top culprits:

Key Mismatch

- **Symptom**: The `kid` from the token header doesn't match any key in your JWKS cache, or you're pulling a stale key during a rotation.
- **Solution**: Log the token's `kid` and verify it exists in the live JWKS endpoint. Ensure your cache has a reasonable TTL (e.g., 24 hours) and a refresh mechanism.

Invalid Signing String

- **Symptom**: The token string contains hidden characters (like `\n` or `\r`) from being read from a file or environment variable. This alters the hash.
- **Solution**: Always use `strings.TrimSpace(tokenString)` before parsing to remove leading/trailing whitespace.

Algorithm Confusion

- **Symptom**: The token was signed with PS256 (PSS padding) but your code is verifying using the default RS256 (PKCS1v15 padding).
- **Solution**: Check the `alg` field in the token header and ensure your JWT library is configured to use the matching verification method (e.g., `jwt.SigningMethodRSAPSS`).

From Theory to Practice: Key Go Implementation Snippets

Client - Exchanging the Code for a Token

```
// In exchangeCodeForToken()
data := url.Values{
    "grant_type":    {"authorization_code"},
    "code":          {code},
    "redirect_uri":  {redirectURI},
    "client_id":     {clientID},
    "code_verifier": {am.codeVerifier}, // The original secret
}
resp, err := http.PostForm(tokenURL, data)
```

The 'proof key' that completes the PKCE flow.

API - Verifying the Token with JWKS

```
// In validateEntraIDToken()
jwt.ParseWithClaims(token, &claims, func(token *jwt.Token)
    (interface{}, error) {
    kid := token.Header["kid"].(string)
    // Fetches key from a thread-safe cache
    publicKey, err := jwksCache.getKey(kid)
    if err != nil {
        return nil, err
    }
    return publicKey, nil // The RSA public key for verification
})
```

Finds the correct public key using the 'kid' from the token header.

Your Production-Ready Checklist

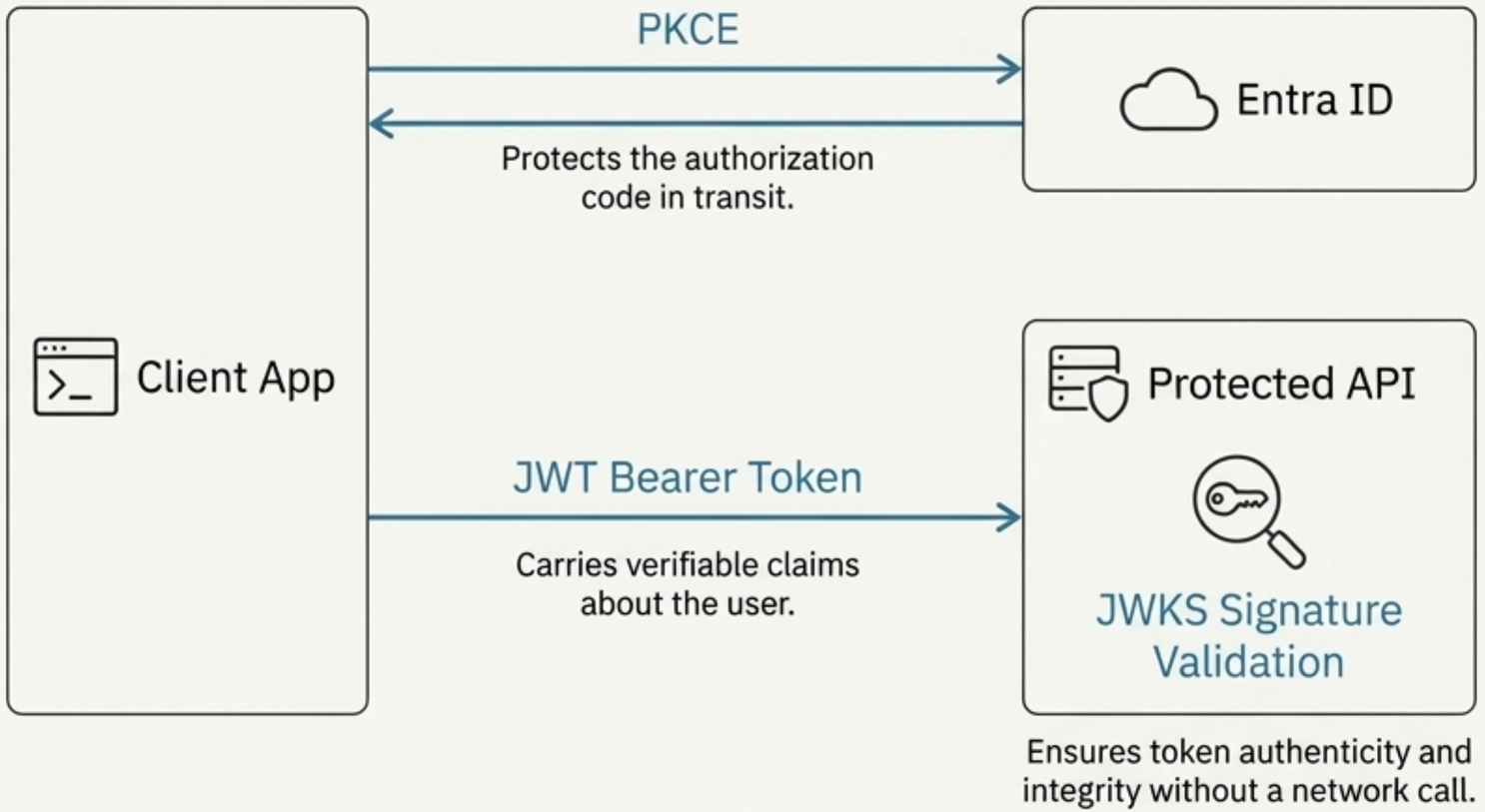
For the Client Application

- **PKCE is Non-Negotiable**: Use it for all public clients.
- **Handle Headless Scenarios**: Implement Device Code Flow for CLI tools.
- **Improve UX**: Automatically open the browser for the user.
- **Be Resilient**: Implement graceful shutdown (SIGINT/SIGTERM).
- **Manage Token Lifecycle**: Automatically refresh tokens before they expire.

For the Protected API

- **Verify Signatures Always**: Use JWKS to validate every incoming token.
- **Cache JWKS Intelligently**: Implement a cache with a TTL (e.g., 24h) and auto-refresh.
- **Enforce HTTPS**: Prevent network-level interception.
- **Validate Standard Claims**: Always check issuer (iss), audience (aud), and expiration (exp).
- **Consider Layered Validation**: For high-security endpoints, verify the token is still active against Microsoft Graph (/me).

The Full Picture: A Chain of Trust from End to End



By combining a dynamic, per-request proof on the client (PKCE) with stateless cryptographic verification on the server (JWKS), we build a system that is secure, scalable, and resilient.